Summary:

Github link: https://github.com/Blue4u0610/SSW567_HW-07b

Results:

In this assignment, I developed a comprehensive suite of 13 unit tests to evaluate a Python program with bugs intended to classify triangles. During the initial execution against the original Triangle.py file, my test suite successfully exposed multiple severe logic flaws, resulting in 10 failed tests (only 3 tests passed, coincidentally due to a hardcoded bug that rejected all inputs). After analyzing the test failures, I rewirte some of the source code to fix 5 bugs. The final execution of the test suite against the improved code resulted in a 100% pass rate.

Metric	Test Run 1 (Before fixing)	Test Run 2 (After fixing)
Tests Planned	13	13
Tests Executed	13	13
Tests Passed	3	13
Defects Found	10	0
Defects Fixed	0	10

Test Strategy:

To determine when a sufficient number of test cases had been established, I employed a combination of Equivalence Partitioning and Boundary Value Analysis to ensure **full statement and branch coverage** without redundant testing.

•**Equivalence Partitioning:** I created test cases for every possible valid output class specified in the requirements (Right, Equilateral, Isoceles, Scalene). For Right and Isoceles, I chose the input order (placing the hypotenuse in the a, b, and c positions) to ensure the logic handled all geometric orientations. I also created partitions for invalid geometric shapes (NotATriangle).

•**Boundary Value Analysis:** I tested the absolute boundaries of the acceptable input ranges. This included testing exactly 0, negative numbers (-1), and numbers just above the maximum limit (201) to rigorously verify the input validation constraints. Once all partitions and extreme boundaries were covered with at least one representative test case, I concluded the test suite was sufficient.

Reflection:

This assignment provided a highly valuable and somewhat unfamiliar perspective on software development. Rather than writing a program from scratch, I was tasked with evaluating and repairing an existing, defective codebase.：

•**What I learned:** I learned that in real-world industry scenarios, developers and testers rarely have the luxury of rewriting code entirely. By first expanding the test suite (TestTriangle.py) to systematically expose defects, and only *then* modifying the source code (Triangle.py) to pass those tests, I experienced a realistic "test-first, patch-later" maintenance workflow. This taught me that robust test cases are not just for validating new features, but serve as an essential safety net when navigating and fixing inherited logic flaws.

•**What worked:** Treating the original buggy program as a "black box" initially worked extremely well. Designing the 13 test cases based strictly on the requirements (Equivalence Partitioning and Boundary Value Analysis) before looking deeply into the faulty logic prevented my tests from being biased by the bad code.

Author: Jinfeng Lan

Assignment Description:

Sometimes you will be given a program that someone else has written, and you will be asked to fix, update and enhance that program. In this assignment you will start with an existing implementation of the classify triangle program that will be given to you. You will also be given a starter test program that tests the classify triangle program, but those tests are not complete.

In order to determine if the program is correctly implemented, you will need to update the set of test cases in the test program. You will need to update the test program until you feel that your tests adequately test all of the conditions. Then you should run the complete set of tests against the original triangle program to see how correct the triangle program is. Capture and then report on those results in a formal test report described below. For this first part you should not make any changes to the classify triangle program. You should only change the test program.

Part 1:

Test ID	Input	Expected Results	Actual Result	Pass or Fail
T-01	(3, 4, 5)	Right	InvalidInput	Fail
T-02	(5, 3, 4)	Right	InvalidInput	Fail
T-03	(4, 5, 3)	Right	InvalidInput	Fail
T-04	(1, 1, 1)	Equilateral	InvalidInput	Fail
T-05	(2, 2, 3)	Isoceles	InvalidInput	Fail
T-06	(2, 3, 2)	Isoceles	InvalidInput	Fail
T-07	(3, 2, 2)	Isoceles	InvalidInput	Fail
T-08	(10, 11, 12)	Scalene	InvalidInput	Fail
T-09	(1, 10, 12)	NotATriangle	InvalidInput	Fail
T-10	(1, 2, 3)	NotATriangle	InvalidInput	Fail
T-11	(-1, 10, 10)	InvalidInput	InvalidInput	Pass
T-12	(0, 10, 10)	InvalidInput	InvalidInput	Pass

Test ID	Input	Expected Results	Actual Result	Pass or Fail
T-13	(201, 10, 10)	InvalidInput	InvalidInput	Pass

Result analysis: 13 tests(10 fail, 3 pass). Because Triangle.py line 34 has b <= b , all possible triangles are incorrectly identifying to InvalidInput.

Part 2:

Test ID	Input	Expected Results	Actual Result	Pass or Fail
T-01	(3, 4, 5)	Right	Right	Pass
T-02	(5, 3, 4)	Right	Right	Pass
T-03	(4, 5, 3)	Right	Right	Pass
T-04	(1, 1, 1)	Equilateral	Equilateral	Pass
T-05	(2, 2, 3)	Isoceles	Isoceles	Pass
T-06	(2, 3, 2)	Isoceles	Isoceles	Pass
T-07	(3, 2, 2)	Isoceles	Isoceles	Pass
T-08	(10, 11, 12)	Scalene	Scalene	Pass
T-09	(1, 10, 12)	NotATriangle	NotATriangle	Pass
T-10	(1, 2, 3)	NotATriangle	NotATriangle	Pass
T-11	(-1, 10, 10)	InvalidInput	InvalidInput	Pass
T-12	(0, 10, 10)	InvalidInput	InvalidInput	Pass
T-13	(201, 10, 10)	InvalidInput	InvalidInput	Pass

Result analysis: 13tests all pass. All bugs in Triangle.py are fixed.

```
bluefile@BluedeMacBook-Air SSW567_HW-07b % python3 -m unittest TestTriangle
.............
----------------------------------------------------------------------
Ran 13 tests in 0.000s

OK
```

Honor pledge:

I pledge my honor that I have abided by the Stevens Honor System.